Jakarta Persistence: ORM и управление данными

Информационные системы · Конспект лекции

ORM и несоответствие моделей

ORM (Object/Relational Mapping) связывает объектную модель приложения с таблицами. Подход Top-Down начинает с классов, Bottom-Up — со схемы БД, Meet-in-the-Middle согласует обе модели. Главные трудности: идентичность объекта и строки, наследование и представление связей. Несколько экземпляров одного класса могут описывать одну строку, поэтому persistence context обеспечивает единую управляемую идентичность в рамках контекста.

Стратегии наследования

Single Table помещает всю иерархию в одну таблицу с дискриминатором: запросы просты и быстры, но много nullable-колонок. Joined Table хранит базовую и дочерние части в отдельных связанных таблицах: схема нормализована, но нужны соединения. Table per Class создаёт таблицу на конкретный класс: удобны ограничения, однако полиморфные запросы и объединение данных сложнее. Выбор зависит от частоты полиморфных запросов, целостности схемы и нагрузки.

Entity и отображение

Сущность объявляется как Entity и имеет устойчивый идентификатор. Маппинг задают аннотациями или XML. Идентификатор может быть простым, составным либо встроенным; генерация ключей передаётся провайдеру/БД. Поля и свойства можно настраивать, исключать временные данные через Transient, задавать enum и порядок загрузки. Сущность должна иметь корректную модель равенства с учётом жизненного цикла идентификатора.

Связи и загрузка

Связи ManyToOne, OneToMany, OneToOne и ManyToMany описывают ассоциации. В двунаправленной связи одна сторона владеет внешним ключом, а inverse side указывает mappedBy; обе стороны объектов в памяти следует синхронизировать. Eager-загрузка сразу получает связь, Lazy откладывает её до обращения. Чрезмерная eager-загрузка создаёт лишние запросы, а lazy-доступ вне открытого контекста может завершиться ошибкой; планируйте выборки явно.

EntityManager и persistence context

EntityManager управляет состояниями сущностей: transient/new, managed, detached и removed. persist добавляет новую сущность, find загружает, remove помечает удаление, merge копирует состояние detached-объекта в управляемый экземпляр. Изменения managed-сущностей отслеживаются автоматически и синхронизируются с БД при flush/commit. Persistence context — не сама БД, а контекст идентичности и отслеживания изменений.

JPQL, Criteria и DAO

JPQL работает с сущностями и их полями, а не напрямую с именами таблиц; Criteria API строит запросы программно. Для производительности важны границы выборки, N+1-запросы, индексы и пакетная обработка. DAO или репозиторий отделяет операции хранения от бизнес-логики; сервис задаёт сценарии и транзакционные границы. DTO используют для передачи выбранных данных без раскрытия всего графа сущностей.

Что запомнить

JPA/Jakarta Persistence — спецификация ORM, а Hibernate и другие провайдеры её реализуют. EntityManager управляет сущностями и контекстом, не является простым JDBC-обёрточным API. Настраивайте владение связью, загрузку и транзакцию осознанно; анализируйте фактические SQL-запросы и не полагайтесь на неявную загрузку.

Термины

ORM · Entity · EntityManager · Persistence Context · JPQL · DAO

Составлено по материалам исходной лекции. Проверьте поколение API и особенности используемой СУБД/контейнера.